

Technical report — NLI over Swiss federal voting booklets with Apertus

What we built, how it works, and what the numbers say. Every number comes from a recorded run in docs/runs/ (short names in brackets, listed under References) on the dev split, the validation set (val) or both; **the test split has not been run**.

- **Track:** Track 2A – OST: Multilingual Natural Language Inference over Swiss Official Voting Booklets
- **Event:** Online
- **Team:** TODO team name — Rashad, Kaan (TODO full names)
- **Demo:** none

1. Summary

Given a claim in German, French or Italian and a source from an official Swiss federal

- `src/llm.py`: the only module that calls a model: one OpenAI-style chat/completions request per case to `BASE_URL` with `API_KEY`, model from `MODEL`, temperature 0. One retry for HTTP 5xx and timeouts, two for HTTP 429. If the endpoint refuses `response_format` (HTTP 400/422) it resends without it; if it refuses the model name it reads `BASE_URL/models` once and takes the Apertus v1.5 8B id (at most three extra requests per run). Tokens of every request are summed into the case's metrics, and every wait counts in its time.
- `src/nli.py` (prompts, versioned by name; answer parsing); `src/context.py` and `src/contexts/` (task A context variants, one file each); `src/booklet.py`, `src/claim_router.py` (parser and router); `src/taskb_context.py` (task B passages).

- **Apertus makes every entailment decision.** The local embedding model (multilingual-e5-small, MIT, open weights, ONNX, baked into the image, CPU via onnxruntime) only chooses which paragraphs Apertus reads.
- **Labels:** the dataset README (checked 2026-10-09) names the classes only: “May be Entailment (0), Unrelated / Neutral (1) or Contradiction (2)”. The prompts use the official guide’s wording (supports, insufficient information, refutes), which agrees with these names.
- **Development tools:** Claude Code wrote code and ran experiments. It is not part of the solution and is never called by the pipeline.

4. Data

- **Dataset:** OSTswiss/MNLIOverSwissVotingBooklets v1.1 (commit 9ff08597, MIT): 1,488 rows, 33

Experiment [run]	Versions	Macro-F1 (all / same backend)	Input tokens	Rule, outcome
Task B long passages, dev [C-dev]	current / B-cut / B-para	0.967 / 0.967 / 0.963 (all one backend)	1,994 / 1,231 / 1,375	B-cut ≤ 0.01 below, ≥ 25 % fewer tokens: passes
B-cut confirmed on val [C-val]	current / B-cut	{}	{}	{}
Task A label errors, dev [D-dev]	current / L1 / L2	{}	{}	{}
(D-val)				
Closed book, dev [E2-cb]	section-route / claim			

section. A length rule would score 0.973 on dev task B but says nothing about the claim; the pipeline never uses it.

- **Evidence pages:** the starter does not check pages; the official scorer might, with an unknown rule.
- **Not political advice.** The system checks a claim against the booklet's text only. Its answers are not voting recommendations. Every task A label 0 or 2 comes with the booklet page and the verbatim text it rests on, so it can be traced to the source booklet.

7. Reproducibility

- **Run:** make run builds the image and answers examples/cases.jsonl; BASE_URL, API_KEY and optionally MODEL from the environment. Own cases: make run CASES=... BOOKLETS=... OUTPUT_DIR=... (README). Dev: python3 scripts/fetch_dev_booklets.py, then make run CASES=data